Alpha-Beta Pruning in Connect 4

Michael Lunn | 13917657

Track B

41052: Advanced Algorithms

Contents

Contents **2**
1. What I Built **3**
1.1 Purpose 3
1.2 Language 3
1.3 Data Representation 3
1.4 Position Evaluation 4
2. The Tool (Track B) **7**
2.1 What does it do? 7
2.2 How the algorithm comes into effect 8
2.4.1 Example 1: Standard game 10
2.4.2 Comparison mode 12
3. What I Learned **14**
3.1 Search speed at depth 14
3.2 Limitations of the window-counting evaluation 14
3.3 The odd-even effect in search depth 15
4. AI Use **16**
4.1 Tools used 16
4.2 What I used it for 16
4.3 Where the AI was wrong or misleading 17
4.4 What I understood vs took on trust 18
5. References **19**

1. What I Built

1.1 Purpose

The purpose of this program is to demonstrate the performance benefits of alpha-beta pruning in a game of Connect 4. It implements a plain minimax algorithm that is used as a reference to validate, and a command-line tool for playing against the engine or comparing the different algorithms.

1.2 Language

The chosen language for the implementation is Python, this was chosen for its readability and relatively easier iteration. It is noted that an implementation in C++ would execute substantially faster, however, the algorithm written in Python would still save the same amount of computations (nodes visited) compared to a standard minimax algorithm and does not change the proportional benefit of the pruning. This does, however, directly limit how deep of a search can be performed in a fixed time budget.

1.3 Data Representation

The Connect Four board is represented as a flat list of 42 integers indexed at row * WIDTH + col. The height of each column is stored in separate list heights[col]. Moves are recorded on a stack (moves), and the search plays and undoes moves on a single mutable Board object rather than copying the board at each recursive call. This is designed to avoid the computation cost of copying a 42-cell board at every node visited per search. A second stack

_won, stores whether a move completed a line of four. This gets computed inside play() and popped inside undo(). This is done so that the search's win check at every node is a constant-time lookup rather than a recomputation.

1.4 Position Evaluation

In order to best understand how the algorithm comes into effect (detailed in Section 2.2), it is important to first grasp how positions on the board are evaluated. Connect Four is won by completing a line of four consecutive cells in a row, column, or diagonal. Typically, Connect Four uses red and yellow discs to distinguish each player's moves, however, in this program character 'X' is used for red, and 'O' is used for yellow for clarity's sake. In a standard 7x6 board there are exactly 69 possible lines of four that can be created. These are referred to as windows.

For each window, the evaluation looks at what is inside that window and scores it according to the below table:

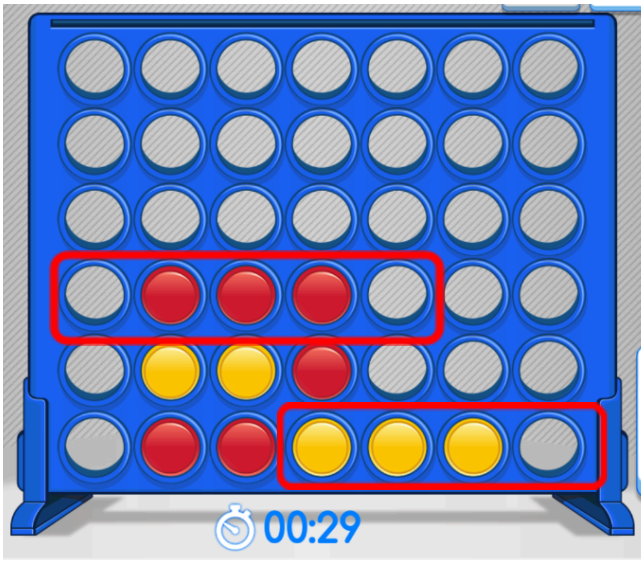
Contents	Meaning	Score
Discs of both X and O	Unwinnable for either player	0
Discs of only X (1, 2, or 3) with the remaining cells empty.	Winnable window for Player X	+1, +5, or +25
Discs of only O (1, 2, or 3) with the remaining cells empty.	Winnable window for Player O	-1, -5, or -25
All empty	Winnable for either player	0

The mapping is assigned as WINDOW_VALUE = (0, 1, 5, 25), indexed by disc count. The values are weighted non-linearly, since a window with three discs of the same type is much more

valuable than a window of only 1 or 2. The total position's evaluation is calculated as the sum of every window's contribution across the whole board.

In addition to the window sum, there is an additional bonus for each disc sitting in the center column (column 4) which adds a flat +3 for X or -3 for O. The center-column bonus is motivated by Victor Allis's 1988 thesis *A Knowledge-Based Approach of Connect Four*, which firstly, generally favours the center column, but secondly, proved that the first player wins with perfect play if and only if their opening move is in the center column. The intention of this bonus is to compensate for an engine's depth-limited search. Allis's result establishes that center control matters over the entire game, however its value may not be realised until many moves later. This causes even a shallow search to bias center play without needing to search deep enough to discover the value itself. However, this effect is not proven, only a plausible rationale offered to justify the design of the evaluation. Even without a center bonus, the existing evaluation function would still prefer center play due to the fact that center cells are included in a greater proportion of total windows.

There is one major limitation in this evaluation function which is that it doesn't take into account if a winning move is actually playable from the current board position. For example, a window of three consecutive discs will be scored the same regardless of how many moves it will take for that player to actually make the winning move.



For example, in the position seen above, red has two separate windows that would be evaluated at +25, whereas yellow only has one. An evaluation of this static position would favour red even though yellow is about to win on the next move. However, the deeper the search goes, the more this gets resolved by the tree itself since a win is scored at 100,000, (far more than any regular board position could score on an evaluation), and if an opponent's winning move is within a search's depth, it will find this and naturally play the blocking move. This means that the evaluation function does not necessarily need to know whether an opponent's window is a threat because it will discover the win a few plies down and play a move to counter it.

2. The Tool (Track B)

2.1 What does it do?

The deliverable is a command-line Connect Four game played against a minimax or alpha-beta engine, including a comparison mode that runs several search configurations on a given position and reports how fast each configuration reaches its decision.

The tool takes a number of arguments, detailed below:

Argument	Default	Description
-d, --depth N	6	Search depth in plies. Higher values look further ahead in the game but take longer to reach a decision.
--algo {minimax, alphabeta}	alphabeta	Search algorithm used by the engine during play. Note: minimax is the unpruned implementation, used to demonstrate the benefits of pruning in the alphabeta pruned version.
--order {left, center, random}	center	Move ordering used by alphabeta algorithm. Left tries columns from left to right (1-7) in order, center tries middle columns first, random chooses randomly (seeded by –seed).
--first {human, computer}	human	Who plays first. Human prompts user first; computer prompts user after first move is played by the computer.
--position	“” (empty board)	Contiguous string of integers representing a sequence of moves to give the tool a starting position.
--seed N	0	Seed used when –order random is selected. Intended so random ordering can be reproduced.
--compare	off	Runs comparison mode instead of a game. Searches a given position (--position) with all four possible configurations and prints nodes visited, cutoffs, timing,

and evaluation for each.

2.2 How the algorithm comes into effect

The CLI is a simple layer over the engine so it does not contain any game logic itself. The CLI's game loop will prompt the user for a move if it is the human's turn or will otherwise construct a Searcher and call search(board, depth). At the root call, Searcher.search loops over the legal moves in the order specified by the user (left, center, random), plays each move on the shared Board, recurses one ply, and undoes it before trying the next.

In the recursive call, the implemented algorithm is negamax alpha-beta, which is a derivative of the standard minimax algorithm. This works on the assumption of a zero-sum property in Connect Four, i.e., one player's gain is exactly equal in magnitude to the opposing player's loss. This means instead of having separate, duplicated logic for maximising and minimising algorithms depending on whose turn it is, every node returns a value from the perspective of whichever player is to move at that node and always tries to maximise it.

In traditional minimax, alpha represents the best score that the maximising player can guarantee so far, while beta represents the best score the minimising player can guarantee so far. These two values' meanings are fixed such that they do not change throughout the search. However, in the negamax form, it is slightly different since there is no longer a fixed maximiser or minimiser. Instead, alpha is the best score the current node can already guarantee for whoever's turn it is to move, while beta is the score the parent node has already secured for itself in the level above. These meanings flip as the search descends down the tree, so this has to be

accounted for when implementing the recursion. This is done by negating alpha and beta and swapping their place in the recursive call.

```
score = -self._alphabeta(board, depth - 1, -beta, -alpha, ply + 1)
```

Also, since the child node is evaluated from the opponent's perspective, its return value is negated to convert it back to the current player's perspective.

At each node, alpha tracks the best score this node can currently guarantee, and beta represents a limit inherited from the parent. Once each child has returned, the node updates its own best score, and updates alpha if it is better than the current alpha value. It then checks whether alpha >= beta. If so, this node has already found something at least as good as what the parent will accept, so the parent will never choose to reach this position regardless of how good the remaining moves might be from the perspective of the current node. As such those moves are skipped entirely, along with their children which is where the pruning is observed.

The effectiveness of the pruning depends heavily on the order that the moves are tried in, since a prune can only occur once a node has found something that is good enough to make its remaining siblings irrelevant. As such, if the strongest move is tried first, alpha rises quickly towards its true best value, so the cutoff condition alpha >= beta is reached sooner and more of the remaining nodes can be pruned. Conversely, if the strongest move is tried last, the node has to work through all the weaker options first which provides comparatively little opportunity for pruning. It is for this reason that there is an option of what order the moves are played in the alphabeta algorithm. Since center cells are valued higher than edge cells, it is expected that moves ordered by center first are likely to result in more pruning.

2.3 Interface decisions

Given the scope of the deliverable was to make the algorithm's behaviour visible rather than to make a polished playing experience, the decision was made to use a CLI rather than a GUI. Node counts, cutoffs, and timing are all valuable metrics that can be calculated and displayed in the terminal rather than creating a dedicated GUI and effectively demonstrate the effect of alphabeta pruning.

While not explicitly a part of the requirements of this implementation, a --compare mode that is entirely separate from the ordinary play was added for two main reasons. First, it gives a direct, visible confirmation that the algorithm behaves as intended, since printing node counts and cutoffs side-by-side across algorithms and move orderings is a much clearer demonstration than only playing against the engine. Second, comparison inherently requires running several searches on the same position, whereas a normal game only ever needs one search per move. Keeping the two separate meant the originary game loop stayed simple and fast, rather than being burdened with computation costs that it doesn't need, that would only be used to support the comparison feature.

2.4 Worked example

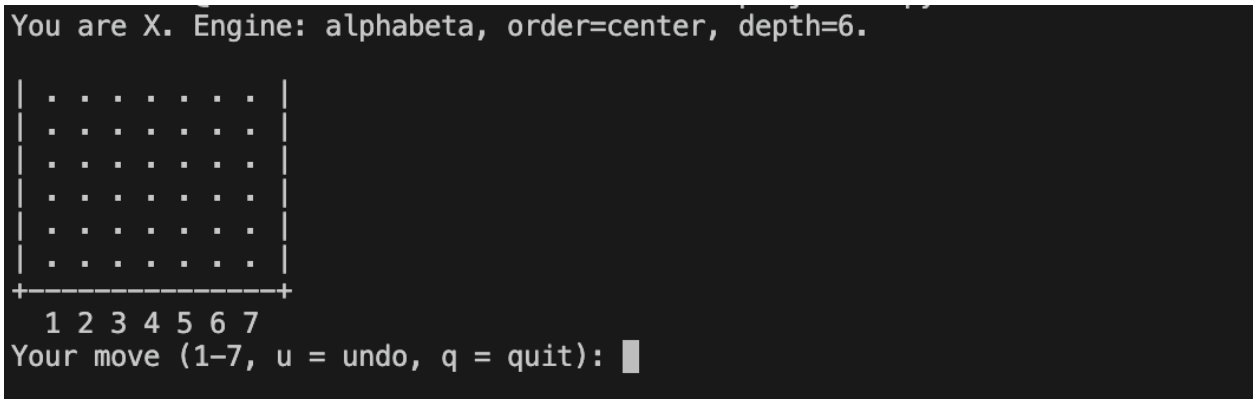
To illustrate the tool's output and what it reveals about the algorithm, this section walks through two runs, a standard game against the engine and a --compare run.

2.4.1 Example 1: Standard game

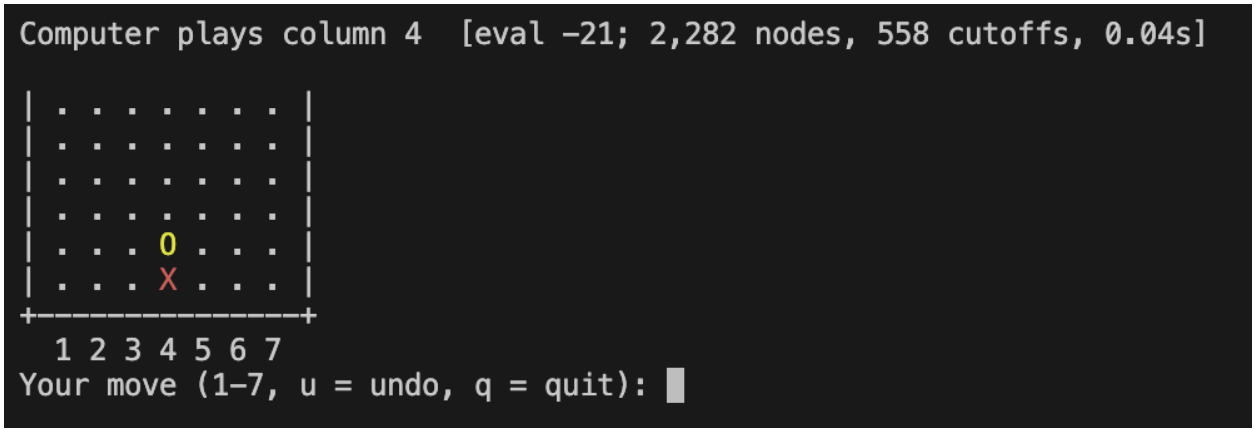
A standard game can be played using the below command:

```
python -m connect4
```

Note: Default values for depth, algorithm, ordering etc. will be used (see table in Section 2.1).



The human plays X and moves first. After each engine move, the CLI prints the board along with the chosen move, its evaluation and the search statistics behind it.



Each time the engine moves, the printed stats are read directly off the SearchResult and Stats objects returned by Searcher.search(). The eval value comes from the search that runs immediately before the computer plays, while the computer is still the side to move. It represents the engine's assessment of its best available move. Here, the computer judged its own position as unfavourable, -21, after an evaluation of depth 6 (default).

```
Computer plays column 3  [eval +63; 2,476 nodes, 799 cutoffs, 0.06s]

| . . . . . . . |
| . . . . . . . |
| . . . . . . . |
| . O O . . . . |
| . X X O . . . |
| X O X X O . . |
+---------------+
  1 2 3 4 5 6 7
Your move (1–7, u = undo, q = quit): 4

| . . . . . . . |
| . . . . . . . |
| . . . . . . . |
| . O O X . . . |
| . X X O . . . |
| X O X X O . . |
+---------------+
  1 2 3 4 5 6 7

Computer plays column 2  [eval forced win in 1; 3,689 nodes, 959 cutoffs, 0.06s]

| . . . . . . . |
| . . . . . . . |
| . O . . . . . |
| . O O X . . . |
| . X X O . . . |
| X O X X O . . |
+---------------+
  1 2 3 4 5 6 7

Computer wins.
```

This example demonstrates the tool functioning as intended for its primary purpose of a playable game where the search happening underneath is made visible move-by-move.

2.4.2 Comparison mode

--compare exists to make the effect of pruning directly measurable, and separate from playing an ordinary game. It searches from a given position with every algorithm/ordering combination and reports how much work each did to arrive at its answer.

E.g.: python -m connect4 --compare --position 4423

```
| . . . . . . . |
| . . . . . . . |
| . . . . . . . |
| . . . . . . . |
| . . . O . . . |
| . X O X . . . |
+---------------+
  1 2 3 4 5 6 7

Searching to depth 6 (X to move)

algorithm  order           value        nodes    cutoffs  time (s)  vs minimax
minimax    left               -9      135,705          0      1.59        1.0x
alphabeta  left               -9       10,013      2,107      0.10       13.6x
alphabeta  random             -9       11,972      2,432      0.12       11.3x
alphabeta  center             -9        3,300        948      0.03       41.1x

All variants agree on the root value.
```

The value here is equivalent to the SearchResult's eval after the search has been completed, it works as a correctness check which confirms that the alpha-beta algorithm reached the same answer as the unpruned minimax reference. The columns that are expected to vary are the measurements. In this specific position and search depth, center-first alpha-beta visits roughly 41x fewer nodes than the standard minimax, and performs significantly better than both left-to-right or randomly ordered moves. This matches the expectation from Section 2.2 that playing stronger moves first (i.e. center columns) raises alpha faster, which triggers the alpha >= beta cutoff sooner and prunes more of the tree.

3. What I Learned

3.1 Search speed at depth

I was surprised that even a depth 8 alpha-beta search from an empty board only took approximately 0.15s with centre-first ordering, visiting only approximately 19,000 nodes compared to the standard minimax's 6.6 million. Initially, in Section 1.2, I outlined the decision to use Python as a trade-off of speed with readability compared to a compiled language like C++ with a mention of how deep a search could be performed in a fixed time budget. This was mainly due to the per-operation speed of C++ compared to Python and the fact that the overhead would concentrate in the nested loop of 69 windows in evaluate(). I think I heavily overestimated how much that difference would have really mattered. I believe that the pruning, not the raw per-node speed, dominates how deep a search could go, with centre-first ordering removing over 99% of the nodes the standard minimax visited. This means Python's slower constant factor per evaluation of the board mattered far less than I expected.

3.2 Limitations of the window-counting evaluation

I was surprised how disconnected the window-counting score can be from what actually matters in Connect Four once I started probing specific positions. A live three on a higher row that is not winnable scores identically to a live three on the bottom row that is winnable. This was acknowledged in the initial position evaluation description in Section 1.4, however, initially I assumed the effect of this would be minimal since a search of sufficient depth would identify a

losing position in its search and play against it but I now believe this is a much larger structural limitation of the evaluation function that could not be controlled by simply adjusting the window weights or scoring bonuses. This connects to a concept discussed in detail in Allis's thesis that has no representation in my current evaluation implementation which is the odd/even parity of potential threats in the game. Depending on what row a live three is on, and whose turn it is to move, the magnitude of the threat can be completely different. I now believe this is the single biggest gap between the program evaluation and actual strength in a given position of Connect Four.

3.3 The odd-even effect in search depth

During my testing of the algorithm, I was surprised to find that the search evaluation doesn't converge smoothly to a single value as I increased the depth, it appears to oscillate in sign depending on whether the depth of the search is odd or even. Running the empty board through depths 1 to 8 showed the below results.

Depth 1: +10	Depth 5: +17
Depth 2: -3	Depth 6: +0
Depth 3: +12	Depth 7: +21
Depth 4: -2	Depth 8: -6

Odd depths are consistently positive while even depths are consistently negative with the only exception being depth 6 which was 0, even so, still a large swing from both depth 5 and depth 7. I initially assumed there had to be a simple bug somewhere in the evaluation but upon further research, I believe this is a well-documented phenomenon called decision oscillation. From my

research, Dana S. Nau first identified it in 1982 in his paper *'The last player theorem'* in which he suggests that biasing may be due to the errors made by the evaluation function. I believe this may point to the same underlying issue in this project as identified earlier, which is that the evaluate() function is flawed, however, this is only a hypothesis. While an in-depth analysis of why this effect is seen with this implementation of the minimax algorithm is out of scope for this assignment, after stumbling upon this discovery, I would be interested in investigating further.

4. AI Use

4.1 Tools used

I used Claude (claude.ai) consistently throughout this project in both the implementation including the initial code scaffold, ongoing debugging and verification, as well as drafting report sections and explaining concepts or outputs I didn't fully understand as I went. I did not use any other AI tools for this project.

4.2 What I used it for

Claude generated the first working version of the repository, including the test suite. Beyond initial generation, I ran various tests on the search code deliberately breaking specific lines and checking what the unit tests would catch. It was also used to help investigate unexpected behaviour I noticed while running the program, e.g. why an evaluation looked wrong or the odd-even effect detailed in Section 3.3.

4.3 Where the AI was wrong or misleading

1. When I asked about an evaluation value of -21 following only a single move from an empty board, Claude speculated that once the opponent played the optimal move in response, “the position should return to something closer to balanced”. I noticed that this wasn’t the case and pushed on this in a later question, Claude ran the actual position and found that this was false. The position stayed unfavourable by a different amount. Claude flagged this as an incorrect claim it had made without checking, which prompted me to be more careful in general about accepting confident-sounding explanations of search values without verifying them against the running code.
2. Claude was not reliably able to explain its own code correctly on the first attempt, even for straightforward control flow. In a worked example describing when the printed eval is computed during a game, Claude initially wrote that it reflected the position “after the computer’s own move,” however, upon inspection of the code, this appeared to be wrong. The code shows the search runs, and result.value is captured before the move is ever played. The eval is the engine’s assessment of the best move made immediately before it commits to it, not an assessment of the resulting position. At this point, I stopped treating Claude’s descriptions of its own generated code as reliable and really went through line-by-line to inspect what the code was doing.

4.4 What I understood vs took on trust

I read and traced through the core search logic (_alphabeta, _minimax, the root loop) myself, including working through the negamax sign convention by hand and verified my understanding by deliberately breaking parts of it and predicting what would happen before checking. I am confident I understand why the algorithm is correct and where its pruning happens and why, partly because I wasn't satisfied with Claude's explanation which prompted me to seek explanation of the alpha-beta algorithm from external sources such as YouTube videos explaining the topic with implementations such in different games.

The effort put in to understand the code was not applied evenly across the codebase, and that was a deliberate choice. I focused almost all of my own verification effort on search.py, since that's where the actual algorithm being demonstrated is. I spent comparatively little time independently checking cli.py beyond confirming it ran correctly and produced sensible output. This applies to similarly to much of the logic in board.py. I think this was a reasonable allocation of effort for this project since the deliverable is specifically about the search algorithm.

I am less confident in my independent understanding of the more theoretical material I introduced in Section 3.3. I relied on Claude's summary of Nau's paper to surmise my findings. I was intentionally vague about any connection between that paper's formal results and my own engine's observed behaviour and have treated it as a hypothesis based on limited understanding rather than something I have personally verified.

5. References

Allis, V. (1988) A Knowledge-based Approach of Connect-Four. *A Knowledge-based Approach of Connect-Four.* [Masters thesis, Vrije Universiteit Amsterdam]

Nau, D. S. (1982). The last player theorem. *Artificial Intelligence*, *18*(1), 53–65. https://doi.org/10.1016/0004-3702(82)90010-8